

Assignment 3

HealthSense234.com receives raw nurse notes on a structured table, but the note text itself is free form (unstructured data). In this assignment, we will use a tiny dataset and a 3-term TF-IDF embedding (3D vectors) to illustrate vector search, then practice IVF/ANN indexing, and finally convert these nurse records to XML with a DTD.

```
Raw table: NurseNote(patient id, address, note text, visit date)
```

```
N1: (101, "Cleveland, OH", "child anxious after vaccination", 2025-02-12)
```

```
N2: (102, "Akron, OH", "persistent cough no fever", 2025-03-18)
```

```
N3: (103, "Columbus, OH", "vaccination follow-up mild fever", 2025-03-20)
```

For this assignment, we build 3D vectors using ONLY the following three terms (in this order):

$t_1 = \text{vaccination}$, $t_2 = \text{anxious}$, $t_3 = \text{cough}$.

TF (term frequency) is the raw count of the term in the note text. IDF uses log base 10:

$$idf(t) = \log \left(\frac{N}{df(t)} \right)$$

where N is the number of documents ($= 3$), and $df(t)$ (document frequency) is the number of documents containing the term at least once.

The TF-IDF vector for a document d is defined as:

$$v(d) = tf(t_1, d) \cdot idf(t_1), tf(t_2, d) \cdot idf(t_2), tf(t_3, d) \cdot idf(t_3)$$

TF-IDF 3D Vectorization for Nurse Notes

a

Compute $df(t_1)$, $df(t_2)$, $df(t_3)$ from the three notes, then compute $idf(t_1)$, $idf(t_2)$, $idf(t_3)$.

✓ Answer ✓

$$df(t_1) = 2$$

$$df(t_2) = 1$$

$$df(t_3) = 1$$

$$idf(t_1) = \log(3/2) = 0.17609125905568124$$

$$idf(t_2) = \log(3/1) = 0.47712125471966244$$

$$idf(t_3) = \log(3/1) = 0.47712125471966244$$

b

Compute the TF–IDF vector for each note N1, N2, N3 as a 3D vector (vaccination, anxious, cough).

✓ Answer

$$v(N1) = q = \langle 0.17609125905568124, 0.47712125471966244, 0 \rangle$$

$$v(N1) = b = \langle 0, 0, 0.47712125471966244 \rangle$$

$$v(N3) = a = \langle 0.17609125905568124, 0, 0 \rangle$$

Similarity Search with Euclidean and Cosine

Use your TF–IDF vectors from Q1. Let:

$$q = v(N1), a = v(N3), b = v(N2).$$

a

Compute Euclidean distances $d(q, a)$, $d(q, b)$ and cosine similarities $\cos(q, a)$, $\cos(q, b)$. Which is closer to q under each measure?

✓ Answer

$$d(q, a) = 0.47712125471966244 \text{ (one dimensional distance)}$$

$$d(q, b) = \sqrt{0.17609125905568124^2 + 2 \cdot 0.47712125471966244^2} = 0.6973503530696353$$

By distance, q is closer to a than b

$$\cos(q, a) = \frac{q \cdot a}{\|q\| \|a\|} = 0.3462415530579614$$

$$\cos(q, b) = \frac{q \cdot b}{\|q\| \|b\|} = 0 \text{ (there is no overlap)}$$

By cos similarity, q is still closer to a than b

b

A clinician issues a search query: “vaccination anxious”. Its 3D embedding (using the same TF–IDF scheme) is:

$$v_{query} = (idf(vaccination), idf(anxious), 0)$$

Rank the top-2 nearest notes among $\{N1, N2, N3\}$ by Euclidean distance and cosine similarity.

✓ Answer

$$v_{query} = v(N1)$$

$$\therefore d(v_{query}, v(N1)) = 0 \text{ and } \cos(v_{query}, v(N1)) = 1$$

Out of a and b , as in part a, a is closest to q .

N1 is closest, with a second of N3.

IVF / ANN Search with k-means (Merged Index + Recall)

Now consider 6 vectors in 2D (the original text is not given):

$P_1(1, 1), P_2(1, 2), P_3(2, 1), P_4(8, 8), P_5(9, 8), P_6(8, 9)$

Two centroids are given: $C_1(1, 1)$ and $C_2(8, 8)$. The distance metric is Euclidean.

a

(k-means assignment step) Assign each P_i to its nearest centroid (C_1 or C_2).

```
[[np.float64(0.0), np.float64(9.899494936611665)],  
 [np.float64(1.0), np.float64(9.219544457292887)],  
 [np.float64(1.0), np.float64(9.219544457292887)],  
 [np.float64(9.899494936611665), np.float64(0.0)],  
 [np.float64(10.63014581273465), np.float64(1.0)],  
 [np.float64(10.63014581273465), np.float64(1.0)]]
```

✓ **Answer**

Points 1, 2, 3 are closer to C_1 and points 4, 5, 6 are closer to C_2

b

Build an IVF view: each centroid corresponds to an inverted list of its assigned points. Give the assignment in a form such as $C_i \leftarrow \{P_j, P_k, P_l\}$.

✓ **Answer**

$C_1 \leftarrow \{P_1, P_2, P_3\}$

$C_2 \leftarrow \{P_4, P_5, P_6\}$

c

Query vector $q = (2, 2)$. For `nprobe=1`, which centroid is probed and which candidate points are searched? Return the top-2 nearest neighbors among the searched candidates.

✓ **Answer**

It is closer to C_1 at a distance of 1.4142135623730951, as compared to 8.48528137423857.

At `nprobe=1`, we search just C_1 .

$\sqrt{2}, 1, 1$ as distances to $P_{1,2,3}$

Top 2 nearest neighbors are $P_{2,3}$.

d

For `nprobe=2` , repeat (c) (probe both centroids).

✓ **Answer**

At `nprobe=2` , we also search C_2 .

It is fairly obvious that $P_{4,5,6}$ are all above 8 units away, much further than $P_{1,2,3}$

Top 2 nearest neighbors are still $P_{2,3}$.

e

Ground-truth check: compute the true top-2 nearest neighbors to q among all six points.

Compute `Recall@2` for `nprobe=1` and `nprobe=2` (`Recall@2` = (# of true top-2 returned)/2) .

✓ **Answer**

We've already calculated the distances to all points, so the closest 2 are still $P_{2,3}$.

Both `nprobe=1` and `nprobe=2` have 100% recall

XML & DTD for the SAME NurseNote data

Consider the XML fragment below:

```
<patient id="101">
  <address>Cleveland, OH</address>
  <visits>
    <visit date="2025-02-12">
      <note>child anxious after vaccination</note>
    </visit>
  </visits>
</patient>
```

And the DTD below:

```
<!ELEMENT patient (address, visits)>
<!ATTLIST patient id ID #REQUIRED>
<!ELEMENT address (#PCDATA)>
<!ELEMENT visits (visit+)>
<!ELEMENT visit (note)>
<!ATTLIST visit date CDATA #REQUIRED>
<!ELEMENT note (#PCDATA)>
```

a

Is the XML valid under this DTD?

✓ **Answer**

Yes, the patient has exactly address and visits in that order, with the id tag.

Address contains text only.

Visits contain one visit, which is compliant.

Each visit does have a date tag and a note within too, with only text within it.

b

Modify both the XML and the DTD to allow an optional element `<hashtags>` with one or more `<hashtag>` children inside `<visit>`, where each hashtag is free text (`PCDATA`), such as “vaccination”, “pediatric”. Give the modified XML snippet and the modified DTD rules.

```
<patient id="101">
  <address>Cleveland, OH</address>
  <visits>
    <visit date="2025-02-12">
      <note>child anxious after vaccination</note>
      <hashtags>
        <hashtag>vaccination</hashtag>
        <hashtag>pediatric</hashtag>
      </hashtags>
    </visit>
  </visits>
</patient>
```

```
<!ELEMENT patient (address, visits)>
<!ATTLIST patient id ID #REQUIRED>
<!ELEMENT address (#PCDATA)>
<!ELEMENT visits (visit+)>
<!ELEMENT visit (note, hashtags?)>
<!ATTLIST visit date CDATA #REQUIRED>
<!ELEMENT note (#PCDATA)>
<!ELEMENT hashtags (hashtag+)>
<!ELEMENT hashtag (#PCDATA)>
```

R&D Practice (One workflow, same dataset) – PostgreSQL or DuckDB

Do this coding practice using either PostgreSQL or DuckDB (your choice). Use the same NurseNote rows *N1* – *N3*.

Deliverables: paste your SQL (or notebook cells) + screenshots/output.

a

Create a table `NurseNote(patient id, address, note text, visit date)` and insert $N1 - N3$.

```
CREATE TABLE NurseNote (  
    patient_id INTEGER PRIMARY KEY,  
    address TEXT,  
    note_text TEXT,  
    visit_date DATE  
);  
  
INSERT INTO NurseNote VALUES  
(101, 'Cleveland, OH', 'child anxious after vaccination', '2025-02-12'),  
(102, 'Akron, OH', 'persistent cough no fever', '2025-03-18'),  
(103, 'Columbus, OH', 'vaccination follow-up mild fever', '2025-03-20');
```

b

Create a second table `NurseNoteVec(patient id, v1, v2, v3)` and insert the TF-IDF vectors you computed in Q1.

```
CREATE TABLE NurseNoteVec (  
    patient_id INTEGER REFERENCES NurseNote(patient_id),  
    v1 DOUBLE PRECISION,  
    v2 DOUBLE PRECISION,  
    v3 DOUBLE PRECISION  
);  
  
INSERT INTO NurseNoteVec VALUES  
(101, 0.1761, 0.4771, 0.0),  
(102, 0.0, 0.0, 0.4771),  
(103, 0.1761, 0.0, 0.0);
```

c

Write one query that computes cosine similarity between $v_{query} = (idf(vaccination), idf(anxious), 0)$ and each note vector, and returns the top-2 notes. (You may implement cosine in SQL directly: a sample code is provided for you to try below.)

```
WITH query_vec AS (  
    SELECT  
        0.1761::DOUBLE PRECISION AS q1,  
        0.4771::DOUBLE PRECISION AS q2,  
        0.0::DOUBLE PRECISION AS q3
```

```

)
SELECT
    n.patient_id,
    (n.v1*q.q1 + n.v2*q.q2 + n.v3*q.q3) /
    (
        sqrt(n.v1*n.v1 + n.v2*n.v2 + n.v3*n.v3)
        *
        sqrt(q.q1*q.q1 + q.q2*q.q2 + q.q3*q.q3)
    ) AS cosine_sim
FROM NurseNoteVec n
CROSS JOIN query_vec q
ORDER BY cosine_sim DESC
LIMIT 2;

```

patient_id	cosine_sim
101	1
103	0.34627025560201363

(2 rows)

d

Export patient 101's record to XML in the structure of Q4 (SQL XML functions or Python is OK) and show validation result (Python lxml or an online validator).

```

SELECT
    xmlelement(
        NAME patient,
        xmlattributes(n.patient_id AS id),
        xmlelement(NAME address, n.address),
        xmlelement(
            NAME visits,
            xmlelement(
                NAME visit,
                xmlattributes(n.visit_date AS date),
                xmlelement(NAME note, n.note_text)
            )
        )
    ) AS patient_xml
FROM NurseNote n
WHERE patient_id = 101;

```

```

<patient id="101">
  <address>Cleveland, OH</address>

```

```
<visit date="2025-02-12">
  <note>child anxious after vaccination</note>
</visit>
</visits>
</patient>
```

Validated on <https://codebeautify.org/xmlvalidator>.

The rest of the sql code along with devshells can be found on:

<https://github.com/tnichols217/postgres-practice>